

Intro to Snort



- ❑ **Snort** is an *open-source network intrusion detection and prevention system (IDS/IPS)* developed by Martin Roesch in 1998
 - ✓ It is widely used for real-time traffic analysis and packet logging on IP networks
 - ✓ Snort can perform protocol analysis, content searching/matching, and can be used to detect a variety of attacks and probes, such as buffer overflows, stealth port scans, CGI attacks, SMB probes, and more

- **Key Features of Snort**

- **Real-Time Traffic Analysis:** Snort monitors network traffic in real-time, analyzing packets as they pass through the network. This allows for immediate detection of suspicious activities

- *Rule-Based Detection: Snort uses a flexible rule-based language to define traffic patterns. These rules can be customized to detect specific types of attacks*
- *Packet Logging: Snort can log packets for later analysis, which is useful for forensic investigations*
- *Protocol Analysis: It can analyze various protocols to detect anomalies and potential threats*
- *Open Source: Being open-source, Snort is highly customizable and has a large community contributing to its development and rule sets*

- **Importance in Network Security**

- *Intrusion Detection: Snort is primarily used to detect intrusions by monitoring network traffic for suspicious patterns. For example, it can detect a Distributed Denial of Service (DDoS) attack by identifying unusual traffic spikes*
- *Prevention: Beyond detection, Snort can also prevent attacks by blocking malicious traffic. For instance, it can block traffic from known malicious IP addresses*
- *Flexibility and Customization: Snort's rule-based system allows network administrators to tailor the detection mechanisms to their specific needs. This flexibility makes it suitable for various network environments*

- *Cost-Effective:* As an open-source tool, Snort provides a cost-effective solution for network security, making it accessible to organizations of all sizes
- *Community Support:* The large community of users and contributors ensures that Snort is continuously updated with new rules and features to address emerging threats

• **Practical Examples**

1. Detecting SQL Injection: A Snort rule can be written to detect SQL injection attempts by looking for specific patterns in HTTP requests. For example:

```
alert tcp any any -> any 80 (msg:"SQL Injection Attempt";  
content:"SELECT"; nocase; http_uri; sid:1000001;)
```

This rule alerts when a SQL SELECT statement is detected in the URI of an HTTP request



2. Blocking Malicious IPs: Snort can be configured to block traffic from known malicious IP addresses. For example:

```
drop ip 192.168.1.100 any -> any any (msg:"Block traffic from malicious IP"; sid:1000002;)
```

This rule drops all traffic from the IP address 192.168.1.100

3. Detecting Port Scans: Snort can detect port scans by monitoring for multiple connection attempts to different ports. For example:

```
alert tcp any any -> any any (msg:"Port Scan Detected"; flags:S; threshold:type both, track by_src, count 20, seconds 10; sid:1000003;)
```

This rule alerts if more than 20 SYN packets are detected from the same source within 10 seconds



- **Tools and Resources**

- *[Snort.org](#): The official website for Snort, where you can download the software, find documentation, and access community resources*
- *[Snort Rules](#): A comprehensive set of rules maintained by the community and available for download to keep your Snort installation up-to-date*
- *[Snorby](#): A web-based interface for managing and analyzing Snort alerts, providing a user-friendly way to interact with Snort data*

Understanding Intrusion Detection Systems (IDS)

- ❑ ***Intrusion Detection Systems (IDS)*** are critical components of network security, designed to monitor network or system activities for malicious activities or policy violations

An IDS can be configured to alert administrators when suspicious activities are detected, allowing for timely responses to potential threats

- ***Types of IDS: Network-based vs. Host-based***

- 1. Network-based IDS (NIDS)***

- ***A Network-based IDS*** monitors network traffic for suspicious activity. It is typically deployed at strategic points within the network, such as at the perimeter or in front of critical servers. NIDS analyze the traffic passing through the entire network segment and can detect a wide range of attacks

Understanding Intrusion Detection Systems (IDS)

- *Example:*

Snort: As a NIDS, Snort can be configured to monitor all incoming and outgoing traffic on a network. It uses predefined rules to detect patterns indicative of malicious activity, such as port scans or SQL injection attempts

- *Advantages:*

- o *Can monitor large networks with minimal performance impact on individual hosts*
- o *Detects attacks that target multiple systems within the network*

Understanding Intrusion Detection Systems (IDS)

- *Disadvantages:*
 - o *May struggle with encrypted traffic*
 - o *Can generate a high number of false positives if not properly tuned*

2. Host-based IDS (HIDS)

- *A Host-based IDS is installed on individual hosts or devices within the network. It monitors the inbound and outbound packets from the device only and alerts the user or administrator of suspicious activity. HIDS can also monitor system logs, file integrity, and other host-specific activities*

Understanding Intrusion Detection Systems (IDS)

- *Example:*

OSSEC: *An open-source HIDS that performs log analysis, integrity checking, Windows registry monitoring, rootkit detection, and real-time alerting*

- *Advantages:*

- o *Provides detailed information about the host's activities*
- o *Can detect insider threats and unauthorized changes to system files*

Understanding Intrusion Detection Systems (IDS)

- *Disadvantages:*

- o *Limited to the host it is installed on*
- o *Can consume significant system resources*

- ***How Snort Fits into the IDS Landscape***

Snort is primarily a Network-based IDS (NIDS), but it also has capabilities that extend into intrusion prevention (IPS)

1. *Real-Time Traffic Analysis: Snort analyzes network traffic in real-time, allowing for immediate detection of suspicious activities. For example, it can detect a SYN flood attack by identifying a large number of SYN packets without corresponding ACK packets*



Understanding Intrusion Detection Systems (IDS)

2. Rule-Based Detection: Snort uses a flexible rule-based language to define traffic patterns. These rules can be customized to detect specific types of attacks. For example, a rule can be written to detect attempts to exploit a known vulnerability in a web application

3. Packet Logging: Snort can log packets for later analysis, which is useful for forensic investigations. For example, if a data breach is suspected, the logged packets can be analyzed to determine the nature and extent of the breach

4. Protocol Analysis: Snort can analyze various protocols to detect anomalies and potential threats. For example, it can detect malformed packets that may indicate an attempt to exploit a protocol vulnerability

Understanding Intrusion Detection Systems (IDS)

- *Practical Examples*

1. Detecting SQL Injection:

```
alert tcp any any -> any 80 (msg:"SQL Injection Attempt";  
content:"SELECT"; nocase; http_uri; sid:1000001;)
```

This rule alerts when a SQL SELECT statement is detected in the URI of an HTTP request, indicating a potential SQL injection attempt

2. Blocking Malicious IPs:

```
drop ip 192.168.1.100 any -> any any (msg:"Block traffic from malicious  
IP"; sid:1000002;)
```

This rule drops all traffic from the IP address 192.168.1.100, effectively blocking a known malicious source



3. Detecting Port Scans:

```
alert tcp any any -> any any (msg:"Port Scan Detected"; flags:S;  
threshold:type both, track by_src, count 20, seconds 10; sid:1000003;)
```

This rule alerts if more than 20 SYN packets are detected from the same source within 10 seconds, indicating a potential port scan

Intro to Snort

Snort Architecture and Components

- ❑ **Snort** is a versatile and powerful tool in network security, known for its ability to detect and prevent intrusions
- Its architecture is modular, allowing for flexibility and customization. Let's break down the key components and modes of Snort

- **Snort Modes**

Snort can operate in three primary modes:

1. Sniffer Mode:

- **Function:** Reads network packets and displays them on the console
- **Use Case:** Useful for real-time monitoring and troubleshooting network issues

Intro to Snort

Snort Architecture and Components

- *Example: Running Snort in sniffer mode can help network administrators see all the traffic passing through a network interface in real-time*

snort -v

This command will start Snort in verbose mode, displaying packet details on the console

2. Packet Logger Mode:

- *Function: Logs packets to disk for later analysis*
- *Use Case: Useful for forensic analysis and detailed traffic logging*

Intro to Snort

Snort Architecture and Components

- *Example: Logging all packets to a specific directory for later review.*

snort -dev -l /var/log/snort

This command logs packets with detailed information to the specified directory

3. Network Intrusion Detection System (NIDS) Mode:

- *Function: Analyzes network traffic against predefined rules to detect and alert on suspicious activities*
- *Use Case: Real-time detection and alerting of potential intrusions*

Intro to Snort

Snort Architecture and Components

- *Example: Running Snort with a specific configuration file to detect intrusions*

snort -c /etc/snort/snort.conf

This command starts Snort using the specified configuration file, which contains the rules and settings for intrusion detection

- **Key Components of Snort**

- 1. Preprocessors:**

- *Function: Preprocessors analyze and modify packets before they reach the detection engine. They handle tasks such as protocol normalization, packet reassembly, and anomaly detection*

Intro to Snort

Snort Architecture and Components

- *Example: The HTTP Inspect preprocessor normalizes HTTP traffic to detect evasion techniques.*

*preprocessor http_inspect: global *

iis_unicode_map unicode.map 1252

This configuration snippet enables the HTTP Inspect preprocessor with specific settings

2. Detection Engine:

- *Function: The core component that applies Snort rules to network traffic to detect suspicious patterns. It compares packet data against a set of predefined rules*

Snort Architecture and Components

- *Example: A rule to detect SQL injection attempts*

*alert tcp any any -> any 80 (msg:"SQL Injection Attempt";
content:"SELECT"; nocase; http_uri; sid:1000001;)*

This rule triggers an alert when a SQL SELECT statement is detected in the URI of an HTTP request

3. Output Modules:

- *Function: Output modules handle the logging and alerting of detected events. They can log to various formats and send alerts to different destinations*

Intro to Snort

Snort Architecture and Components

- *Example: Configuring Snort to log alerts to a file and send them to a remote syslog server*

output alert_fast: alert.ids

output alert_syslog: LOG_AUTH LOG_ALERT

These configuration lines set up Snort to log alerts to a file named alert.ids and send alerts to the syslog server



Ali Ali

Snort Architecture: Snort Rule Syntax and Usage

- ❑ **Snort's** architecture is designed to be modular and flexible, allowing it to efficiently detect and respond to network threats

Understanding Snort rule syntax and usage is crucial for effectively leveraging its capabilities

- **Understanding Rule Structure**

A Snort rule consists of two main parts: the rule header and the rule options

1. Rule Header

The rule header defines the action to be taken, the protocol, source and destination IP addresses, ports, and the direction of traffic

*action protocol source_ip source_port -> destination_ip destination_port
(options)*



Snort Architecture: Snort Rule Syntax and Usage

- *Action: What Snort should do when it matches the rule (e.g., alert, log, pass, drop)*
- *Protocol: The protocol to match (e.g., TCP, UDP, ICMP)*
- *Source IP/Port: The source IP address and port*
- *Direction: The direction of traffic (-> for unidirectional, <> for bidirectional)*
- *Destination IP/Port: The destination IP address and port*

Snort Architecture: Snort Rule Syntax and Usage

- **Rule Options**

Rule options provide additional criteria for matching packets and specify what information to log or alert on. They are enclosed in parentheses and separated by semicolons

1. Writing Basic Snort Rules

Let's look at some examples to understand how to write basic Snort rules

Example 1: Detecting ICMP Echo Requests (Ping)

*alert icmp any any -> any any (msg:"ICMP Echo Request Detected";
itype:8; sid:1000001;)*

- **Action:** alert - Generate an alert
- **Protocol:** icmp - Match ICMP packets

Snort Architecture: Snort Rule Syntax and Usage

- *Source/Destination: any any* - Match any IP address and port
- *Options:*
 - o *msg* - Message to display when the rule matches
 - o *itype* - ICMP type (8 for echo request)
 - o *sid* - Snort ID for the rule

Example 2: Detecting HTTP GET Requests

*alert tcp any any -> any 80 (msg:"HTTP GET Request Detected";
content:"GET "; http_method; sid:1000002;)*

- *Action: alert*
- *Protocol: tcp*

Snort Architecture: Snort Rule Syntax and Usage

- *Source: any any*
- *Destination: any 80 - Match traffic to port 80 (HTTP).*
- *Options:*
 - o *msg - Message to display*
 - o *content - Match the content "GET "*
 - o *http_method - Specify that the content should be in the HTTP method field*
 - o *sid - Snort ID*

Snort Architecture: Snort Rule Syntax and Usage

- **Rule Options and Actions**

Snort rule options can be categorized into several types:

1. General Options: Provide context for the rule

- *msg*: Message to display when the rule matches
- *sid*: Unique Snort ID for the rule
- *rev*: Revision number of the rule

2. Payload Options: Specify criteria based on the packet payload

- *content*: Match specific content in the packet
- *nocase*: Case-insensitive content match
- *offset*: Start searching for content at a specific byte offset

3. Non-Payload Options: Criteria not based on the payload

- *ttl: Match packets with a specific Time-To-Live value*
- *tos: Match packets with a specific Type of Service value*
- *id: Match packets with a specific IP ID*



4. Post-Detection Options: Actions to take after a match

- *logto*: Log the packet to a specific file
- *session*: Log the entire session

Example 3: Blocking Malicious IPs

drop ip 192.168.1.100 any -> any any (msg:"Block traffic from malicious IP"; sid:1000003;)

- *Action: drop - Drop the packet*
- *Protocol: ip*
- *Source: 192.168.1.100 any - Match traffic from IP 192.168.1.100*
- *Destination: any any*
- *Options:*
 - o *msg - Message to display*
 - o *sid - Snort ID*

Intro to Snort

Advanced Rule Writing

- ❑ **Advanced rule writing** in Snort involves using variables, advanced options, and crafting rules to detect specific attack types

This allows for more precise and efficient detection of threats

- **Using Variables and Advanced Options**

1. Variables

- Variables in Snort rules help to make the rules more readable and easier to manage. They are defined in the Snort configuration file (*snort.conf*) and can be used to represent IP addresses, ports, or other values

Intro to Snort

Advanced Rule Writing

- *Example: Defining and using variables*

Defining Variables:

```
var HOME_NET 192.168.1.0/24
```

```
var EXTERNAL_NET any
```

```
var HTTP_PORTS 80
```

2. Using Variables in Rules:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET $HTTP_PORTS  
(msg:"External to HTTP"; sid:1000004;)
```

This rule uses the variables to match any TCP traffic from external networks to the HTTP port on the home network

Intro to Snort

Advanced Rule Writing

- **Advanced Options**

- *Advanced options in Snort rules provide more granular control over the detection criteria. These options include content matching, regular expressions, and flow control*
- *Example: Using advanced options*

1. Content Matching:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET $HTTP_PORTS  
(msg:"Possible XSS Attack"; content:"<script>"; nocase; sid:1000005;)
```

This rule detects potential cross-site scripting (XSS) attacks by looking for the <script> tag in HTTP traffic

Intro to Snort

Advanced Rule Writing

2. Regular Expressions:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET $HTTP_PORTS  
(msg:"Possible SQL Injection"; pcre:"/select.*from/i"; sid:1000006;)
```

This rule uses a Perl Compatible Regular Expression (PCRE) to detect SQL injection attempts by matching the pattern select ... from

3. Flow Control:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET $HTTP_PORTS  
(msg:"HTTP POST Request"; flow:to_server,established; content:"POST";  
http_method; sid:1000007;)
```

This rule detects HTTP POST requests by ensuring the traffic is flowing to the server and the connection is established

Intro to Snort

Advanced Rule Writing

- **Writing Rules for Specific Attack Types**

1. Detecting Buffer Overflow Attacks

- *Buffer overflow attacks can be detected by looking for unusually long strings in certain fields*

- **Example:**

```
alert tcp $EXTERNAL_NET any -> $HOME_NET 21 (msg:"FTP Buffer  
Overflow Attempt"; flow:to_server,established; content:"USER ";  
pcre:"/^USER\s[a-zA-Z0-9]{100,}/"; sid:1000008;)
```

This rule detects potential buffer overflow attempts in FTP USER commands by looking for usernames longer than 100 characters

2. Detecting Port Scans

- *Port scans can be detected by monitoring for multiple connection attempts to different ports from the same source*

- *Example:*

alert tcp any any -> \$HOME_NET any (msg:"Port Scan Detected"; flags:S; threshold:type both, track by_src, count 20, seconds 10; sid:1000009;)

This rule alerts if more than 20 SYN packets are detected from the same source within 10 seconds, indicating a potential port scan

3. Detecting Malware Communication

- *Malware often communicates with command and control (C&C) servers. This can be detected by looking for known malicious domains or IP addresses*

- *Example:*

alert udp \$HOME_NET any -> \$EXTERNAL_NET 53 (msg:"DNS Query to Malicious Domain"; content:"badmalware.com"; nocase; sid:1000010;)

This rule detects DNS queries to a known malicious domain

Intro to Snort

Preprocessors and Their Configuration

❑ **Preprocessors** in Snort are essential components that analyze and modify packets before they reach the detection engine

They handle tasks such as protocol normalization, packet reassembly, and anomaly detection, enhancing Snort's ability to detect sophisticated attacks

- *Types of Preprocessors*

- *HTTP Inspect*: Normalizes HTTP traffic to detect evasion techniques
- *Frag3*: Reassembles fragmented IP packets
- *Stream5*: Reassembles TCP streams and tracks session states
- *SMTP Preprocessor*: Analyzes SMTP traffic for email-based attacks
- *DNS Preprocessor*: Analyzes DNS traffic for anomalies and potential attacks



Intro to Snort Preprocessors and Their Configuration

- **Configuring and Tuning Preprocessors for Optimal Performance**

1. HTTP Inspect Preprocessor

- *The HTTP Inspect preprocessor normalizes HTTP traffic, making it easier to detect evasion techniques used by attackers*
- *Configuration Example:*

```
preprocessor http_inspect: global \  
    iis_unicode_map unicode.map 1252  
preprocessor http_inspect_server: server default \  
    profile all ports { 80 8080 8180 } \  
    oversize_dir_length 500
```

Intro to Snort

Preprocessors and Their Configuration

- *global*: Sets global options for HTTP Inspect
- *iis_unicode_map*: Specifies the Unicode map file
- *server default*: Configures the default server profile
- *profile all*: Applies all inspection profiles
- *ports*: Specifies the ports to inspect
- *oversize_dir_length*: Sets the maximum directory length

2. Frag3 Preprocessor

- The Frag3 preprocessor reassembles fragmented IP packets, which is crucial for detecting attacks that use fragmentation to evade detection
- Configuration Example:

Intro to Snort

Preprocessors and Their Configuration

preprocessor frag3_global: max_fragments 65536

preprocessor frag3_engine: policy first detect_anomalies

- *max_fragments*: Sets the maximum number of fragments to track
- *policy first*: Uses the first fragment policy for reassembly
- *detect_anomalies*: Enables anomaly detection

3. Stream5 Preprocessor

- *The Stream5 preprocessor reassembles TCP streams and tracks session states, which is essential for detecting attacks that span multiple packets*



Intro to Snort

Preprocessors and Their Configuration

- *Configuration Example:*

preprocessor stream5_global: track_tcp yes, track_udp yes, track_icmp no

preprocessor stream5_tcp: policy windows, use_static_footprint_sizes

- *track_tcp: Enables TCP tracking*
- *track_udp: Enables UDP tracking*
- *track_icmp: Disables ICMP tracking*
- *policy windows: Uses the Windows policy for TCP reassembly*
- *use_static_footprint_sizes: Optimizes memory usage*

Intro to Snort Preprocessors and Their Configuration

- **Practical Examples**

1. Detecting HTTP Evasion Techniques:

```
preprocessor http_inspect: global \  
    iis_unicode_map unicode.map 1252  
preprocessor http_inspect_server: server default \  
    profile all ports { 80 8080 8180 } \  
    oversize_dir_length 500
```

This configuration normalizes HTTP traffic and inspects common HTTP ports, making it easier to detect evasion techniques

Intro to Snort Preprocessors and Their Configuration

2. Reassembling Fragmented Packets:

preprocessor frag3_global: max_fragments 65536

preprocessor frag3_engine: policy first detect_anomalies

This configuration reassembles fragmented packets and detects anomalies, which is crucial for identifying fragmentation-based attacks

3. Tracking TCP Sessions:

preprocessor stream5_global: track_tcp yes, track_udp yes, track_icmp no

preprocessor stream5_tcp: policy windows, use_static_footprint_sizes

This configuration tracks TCP and UDP sessions, using the Windows policy for TCP reassembly, which helps in detecting multi-packet attacks



Intro to Snort

Snort Challenge - The Basics

❑ **The Snort Challenge** is a practical exercise designed to *help you apply your Snort skills in real-world scenarios*

It involves writing Snort rules to analyze network traffic and detect malicious activities. Let's break down the key components of the challenge

- **Introduction to the Challenge**

- *The Snort Challenge typically involves a series of tasks where you are given network traffic data (often in the form of pcap files) and asked to write Snort rules to detect specific types of attacks*
- *The goal is to simulate real-world scenarios where you need to identify and respond to network threats*

Intro to Snort

Snort Challenge - The Basics

Simulated Attack Scenarios

In the Snort Challenge, you might encounter various simulated attack scenarios. Here are a few examples:

1. HTTP Attacks:

- *Scenario: Detecting SQL injection attempts in HTTP traffic*
- *Example Rule:*

```
alert tcp $EXTERNAL_NET any -> $HOME_NET 80 (msg:"SQL Injection Attempt"; content:"SĒLECT"; nocase; http_uri; sid:1000011;)
```

This rule alerts when a SQL SELECT statement is detected in the URI of an HTTP request

Intro to Snort

Snort Challenge - The Basics

2. FTP Attacks:

- *Scenario: Detecting brute force login attempts on an FTP server*
- *Example Rule:*

```
alert tcp $EXTERNAL_NET any -> $HOME_NET 21 (msg:"FTP Brute Force Attempt"; flow:to_server,established; content:"530 Login incorrect"; sid:1000012;)
```

This rule alerts when multiple failed login attempts are detected on an FTP server

Intro to Snort

Snort Challenge - The Basics

3. DNS Attacks:

- *Scenario: Detecting DNS queries to known malicious domains.*
- *Example Rule:*

alert udp \$HOME_NET any -> \$EXTERNAL_NET 53 (msg:"DNS Query to Malicious Domain"; content:"badmalware.com"; nocase; sid:1000013;)

This rule alerts when a DNS query to a known malicious domain is detected

Intro to Snort

Snort Challenge - The Basics

- **Analyzing and Responding to Alerts**

Once you have written and deployed your Snort rules, the next step is to analyze the alerts generated by Snort and respond appropriately

Analyzing Alerts

1. Reviewing Alert Logs:

- *Snort logs alerts to a specified file or sends them to a remote server. Reviewing these logs helps you understand the nature of the detected threats*

- **Example:**

output alert_fast: alert.ids

output alert_syslog: LOG_AUTH LOG_ALERT

This configuration logs alerts to a file named alert.ids and sends them to the syslog server



Intro to Snort

Snort Challenge - The Basics

2. Using Analysis Tools:

- Tools like Snorby or BASE (Basic Analysis and Security Engine) can help visualize and analyze Snort alerts
- **Example:** Using Snorby to view and analyze alerts

snorby -u admin -p password

This command logs into Snorby with the specified credentials to view and analyze alerts

• Responding to Alerts

1. Investigating Alerts:

- Investigate the source and nature of the alert to determine if it is a false positive or a genuine threat



Ali Ali

Intro to Snort

Snort Challenge - The Basics

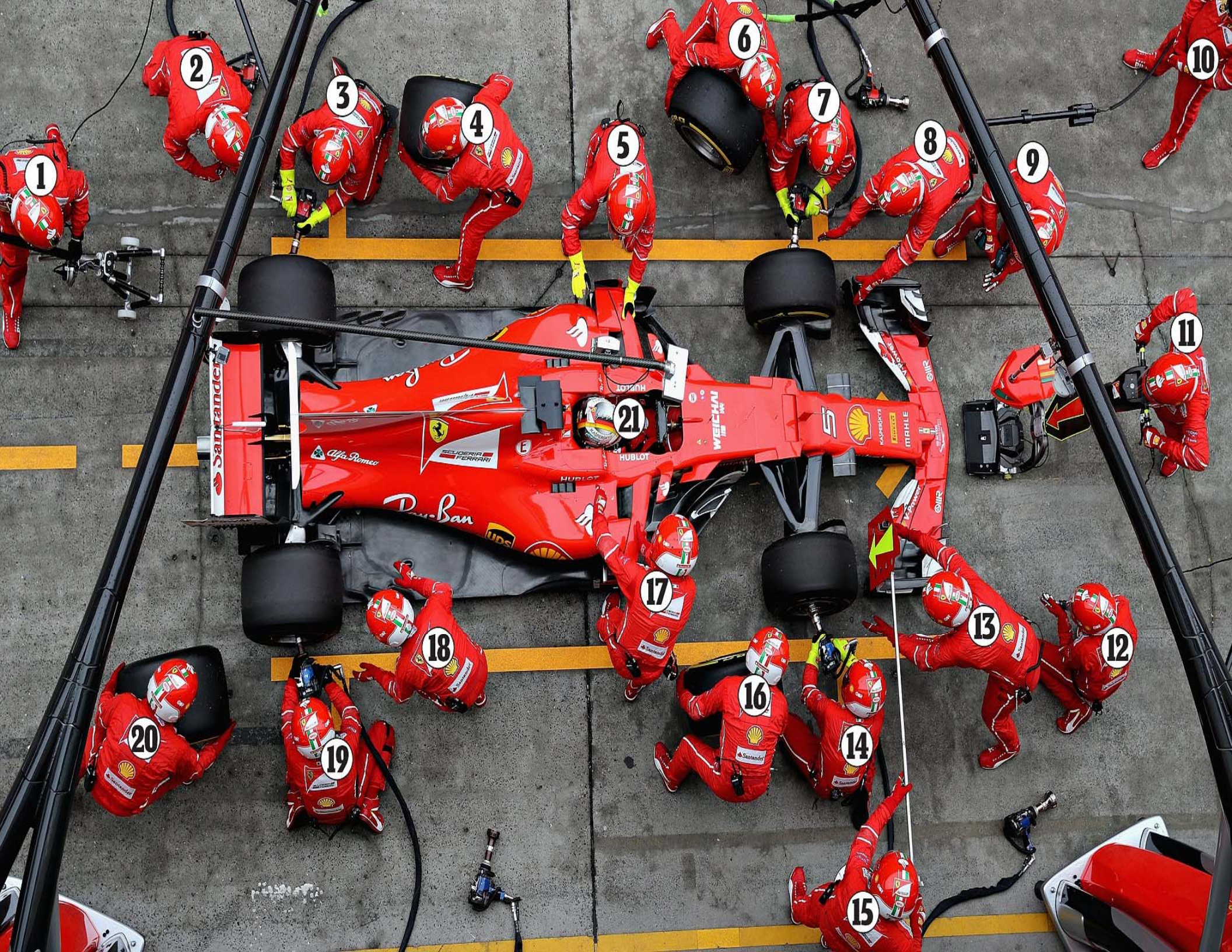
- *Example: Analyzing a detected SQL injection attempt to identify the source IP and the targeted web application*

2. Mitigating Threats:

- *Take appropriate actions to mitigate the detected threat, such as blocking malicious IP addresses or updating firewall rules*
- *Example: Blocking a malicious IP address detected by Snort*

iptables -A INPUT -s 192.168.1.100 -j DROP

This command blocks all traffic from the IP address 192.168.1.100



Questions

THANK YOU!



Ali Ali

Ali Ali